Introdução ao Canvas 2D e Three.js (Extra)

Primeiros passos com renderização no navegador

Pedro Ximenes

Universidade Católica de Pernambuco (UNICAP)
Computação Gráfica

1 de março de 2026

Como esta aula está organizada

Ordem da conversa

- Primeiro: **Canvas 2D**.
- Depois: **como executar** os exemplos.
- Em seguida: **Three.js**.
- No final: **comparação** entre os dois.

Ideia central

Nesta disciplina, vamos começar pelo que nos deixa **ver cada passo do desenho**. Por isso, o ponto de partida é o Canvas 2D.

Por que usar JavaScript nesta disciplina?

JavaScript

Menor curva de aprendizagem, basta um navegador e um arquivo `.html`.

- Não exige **instalar compilador**.
- O resultado aparece na tela imediatamente.
- O código, a lógica e a imagem ficam no mesmo ambiente.
- Facilita testar, errar, ajustar e ver o efeito.

Canvas 2D: o que ele nos entrega?

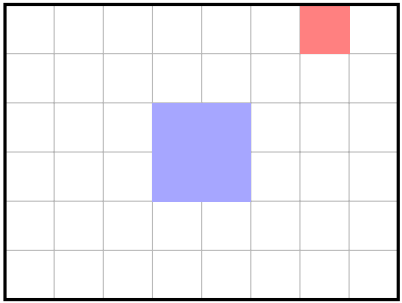
Definição

Canvas 2D é uma API de desenho no navegador.

O `<canvas>` é um elemento HTML que reserva uma área retangular na página onde você pode desenhar via JavaScript: pixels, linhas, formas, imagens, animações.

Leitura correta

Quando usamos Canvas 2D, estamos desenhando **quadro a quadro** em uma área retangular.



grade de pixels

Primeiro exemplo: criando um canvas

Passo 1

Criamos a área de desenho no HTML.

```
<!-- Cria um elemento canvas de 600 px de largura por 300 de altura -->
<canvas id="tela" width="600" height="300"></canvas>
<script>
  const canvas = document.getElementById("tela");
  <!-- Abre o contexto de renderizacao 2D do canvas. -->
  <!-- ctx eh o objeto com todos os metodos de desenho -->
  const ctx = canvas.getContext("2d");
</script>
```

`width` e `height` definem o tamanho real da área de desenho.

Dica:

teste rapidamente o código usando um compilador online (`www.onecompiler.com/html`)

O que é o `ctx`?

Definição

`ctx` significa **contexto de renderização**.
É o objeto que guarda os comandos de desenho do Canvas 2D.

- `ctx.fillRect(...)` desenha um retângulo.
- `ctx.clearRect(...)` limpa uma área.
- `ctx.beginPath()`, `moveTo()`, `lineTo()` constroem linhas.
- `ctx.fillStyle` e `ctx.strokeStyle` controlam cor.

Exemplo pequeno: desenhando um retângulo

```
const canvas = document.getElementById("tela");
const ctx = canvas.getContext("2d");

ctx.fillStyle = "steelblue";
ctx.fillRect(40, 30, 120, 80);
```

Como ler isso

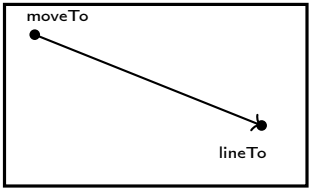
- (40, 30): ponto inicial (canto superior esquerdo)
- 120: largura (width)
- 80: altura (height)

Exemplo pequeno: desenhando uma linha

```
ctx.beginPath(); // Inicia um novo caminho (limpa caminho anterior)
ctx.moveTo(20, 20); // Move o "cursor" para o ponto inicial da linha
ctx.lineTo(180, 100); // Traca uma linha ate o ponto (180, 100)
ctx.stroke(); // Renderiza efetivamente o caminho na tela com a cor padrao
```

Ideia

A linha não aparece enquanto só definimos o caminho.
Ela aparece quando chamamos `stroke()`.



Pensando em rasterização

O que interessa para Computação Gráfica

Mesmo usando funções prontas, a imagem final continua sendo feita de pixels.

- Uma reta matemática é contínua.
- A tela é discreta.
- Em algum momento, a reta precisa virar pixels acesos.

* O Canvas 2D nos ajuda a visualizar o processo de converter imagens baseadas em vetores em pixels na tela (**rasterização**).

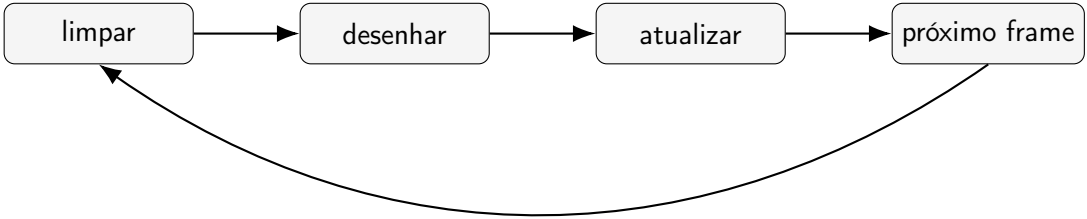
Loop de animação: a estrutura mínima

```
let x = 0; // posicao horizontal do quadrado

function desenhar() {
  ctx.clearRect(0, 0, canvas.width, canvas.height); // apaga todo o canvas
  ctx.fillRect(x, 40, 40, 40); // desenha quadrado 40 por 40
  x += 2; // avanca 2px por frame
  requestAnimationFrame(desenhar); // agenda o proximo frame (~60 vzs/seg)
}

desenhar(); // dispara o loop
```

Leitura

Limpa a tela, desenha o novo quadro, atualiza o estado e agenda o próximo quadro.

Como enxergar o loop



Regra prática

Animação é repetição controlada: o quadro muda porque os dados mudam a cada iteração.

Como rodar os exemplos

Opção recomendada

Abrir a pasta `cg/` no VS Code e usar a extensão **Live Server**.

1. Abra a pasta do projeto.
2. Clique com o botão direito no arquivo `.html`.
3. Escolha *Open with Live Server*.

Outras formas

- `python -m http.server 8080`
- Dois cliques no `.html`, quando o exemplo não depender de módulos ou carregamentos externos.

Estrutura básica do projeto cg/

```
cg/
  README.md
  rasterizacao/
    dda.html
    bresenham_reta.html
    bresenham_circulo.html
  curvas/
    bezier.html
  transformacoes2d/
    translacao.html
    rotacao.html
    escala.html
    composicao.html
  viewing/
    janela_viewport.html
```

Começamos com `rasterizacao/` e `curvas/`. Depois avançamos para `transformacoes2d/` e `viewing/`.

Quando o Canvas 2D começa a ficar limitado?

Sinal de alerta

Quando a cena fica mais complexa, passamos a gastar tempo demais com a estrutura e pouco com o conceito.

- Câmera.
- Objetos 3D.
- Luz.
- Materiais.
- Transformações encadeadas.

É nesse ponto que entra o Three.js.

Three.js: ideia geral

Definição

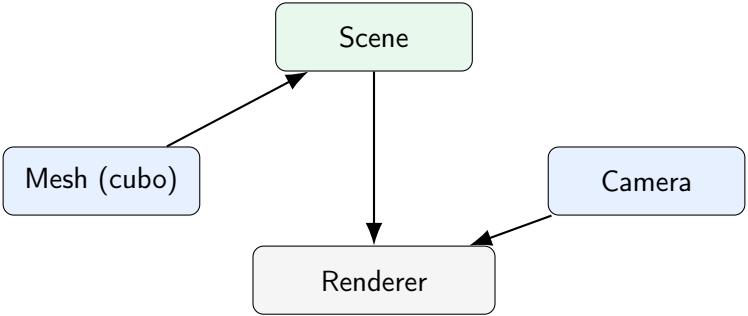
Three.js é uma biblioteca de mais alto nível para montar cenas gráficas no navegador.

- Organiza a cena em objetos.
- Oferece câmera, geometria, material e renderizador.
- Permite focar mais na **cena** e menos na infraestrutura.

Exemplo pequeno: a menor cena em Three.js

```
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, 1.6, 0.1, 1000);
const renderer = new THREE.WebGLRenderer();

const geometry = new THREE.BoxGeometry();
const material = new THREE.MeshBasicMaterial({ color: 0x44aa88 });
const cube = new THREE.Mesh(geometry, material);

scene.add(cube);
camera.position.z = 3;
renderer.render(scene, camera);
```

O que esse código está montando?



Resumo

Em Three.js, pensamos em **quem está na cena**, **quem observa** e **quem desenha**.

Animação simples em Three.js

```
function animar() {
  cube.rotation.y += 0.01;
  renderer.render(scene, camera);
  requestAnimationFrame(animar);
}

animar();
```

Observe

O loop continua existindo.
A diferença é que agora a biblioteca já cuida de boa parte da renderização.

Canvas 2D x Three.js

Canvas 2D

- Mais próximo do desenho manual.
- Excelente para enxergar pixels, traços e etapas.
- Ideal para introduzir rasterização e lógica de animação.

Three.js

- Mais próximo da montagem de uma cena.
- Reduz código estrutural.
- Útil quando queremos trabalhar com objetos e transformações com mais rapidez.

Como vamos usar na disciplina

Primeiro, Canvas 2D para entender os fundamentos.
Depois, Three.js para acelerar construções mais ricas.

Fechamento

O que deve ficar claro hoje

- **ctx** é a porta de entrada para desenhar no Canvas 2D.
- Toda animação básica segue um loop.
- JavaScript é a escolha mais simples para começar.
- Three.js entra quando a cena pede mais estrutura.

Na próxima etapa, podemos abrir um exemplo e desenhar pixel por pixel.